Navigating the Privacy–Robustness–Performance Trilemma in Federated Learning

Jonas Müller
7399328
jonas.mueller-1@studium.uni-hamburg.de

Rebekka Schnoor
7159351
rebekka.schnoor@studium.uni-hamburg.de

July 14, 2026

[2–4 sentences summarizing the paper: motivation (the trilemma), what we did (8-configuration empirical study on MNIST/CIFAR-10 with Flower+Opacus+BLADES), key finding (compounding behavior, direction of magnitude), and main take-away for practitioners.]

Contents

List of abbreviations

FL Federated Learning

IID independent and identically distributed

DP-SGD Differentially Private Stochastic Gradient Descent

SGD Stochastic Gradient Descend

DP Differential Privacy

[glossary]

1 Introduction

We present a systematic empirical study of the Privacy–Robustness–Performance trilemma in Federated Learning (FL), examining how simultaneously deployed defenses interact and compound across all three axes.

[Summarize results and contributions.]

1.1 Background and Motivation

In the past ten years, FL has emerged as a framework for achieving distributed training without the need for data centralization. This is of particular interest when processing sensitive data, such as in healthcare or mobile applications. The local training allows to leave the data with the clients but opens new attack surfaces for model poisoning during training through the transmission of manipulated updates. Also, the transmitted gradients can still leak information about the used training data set. Additionally, gradient transmission at each round causes significant communication cost, especially for complex models.

In an effort to fortify the resilience of models and safeguard client privacy, numerous methodologies have been implemented in recent years. Robustness-enhancing methods are designed to detect malicious updates by assuming a significant deviation from the honest majority. Privacy-preserving algorithms add a mask or calibrated noise to the local updates before transmitting them to the central server.

1.2 Problem Statement

Ensuring model robustness and client privacy while balancing communication cost and model accuracy has been identified as a significant challenge. Adding privacy-enhancing noise to individual updates reduces a malicious server's ability to extract information about local datasets, while simultaneously reducing model accuracy. Discarding unusual updates improves robustness against Byzantine adversaries but risks disregarding legitimate updates, particularly in settings where data is not independent and identically distributed (IID). Compression techniques that reduce transmission load limit the information transmitted to the server and thus reduce model accuracy.

In 2023, Allouah et al. [1] proved Theorem 1, stating that the combination of such methods yields a lower bound for the expected training error that exceeds the sum of the individual cost for robustness and privacy by a multiplicative term.

Theorem 1. *Consider distributed mean estimation in $\mathbb{R}^d$ with n users, of which an η-fraction may behave maliciously. Any algorithm, producing an estimate $\hat{\mu}$ of the true mean $\mu \in \mathbb{R}^d$ of inputs, and achieving (ε, δ)-local DP and robustness to η-corruption must incur an estimated error*

$$\mathbb{E} \parallel \hat{\mu} - \mu \parallel_2^2 = \tilde{\Omega}\left(\frac{\eta}{\varepsilon^2} + \frac{d}{\varepsilon^2 n} + \eta G^2\right). \tag{1}$$

A position paper by the same group [2] highlights a gap between the theoretical lower bound and current implementations.

1.3 Research Questions

The project is structured around three research questions:

1. **Isolation:** How do differential privacy, Byzantine-robust aggregation, and gradient compression individually affect model accuracy, robustness, and computational efficiency?
2. **Combination:** How do pairwise and full combinations of the three mechanisms interact? Do the costs compound additively, sub-additively, or super-additively?
3. **Trust and Privacy:** How do the trust scores assigned by FLTrust evolve throughout training, and what do they reveal about the interaction between privacy, robustness, and model performance?

1.4 Contributions

[Bullet list mirroring Section 6 of the Exposé: (i) systematic 8-configuration mapping on MNIST/CIFAR-10 with joint trilemma reporting; (ii) reproduction and cross-framework extension of BLADES baselines; (iii) initial test of joint clipping-norm calibration as a mitigation heuristic; (iv) open-source reproducible pipeline.]

1.5 Structure of the Report

The remainder of the report is organized as follows. Section 2 provides the theoretical background of all three trilemma axes and derives the joint lower bound. Section 3 defines the threat model and evaluation metrics. Section 4 describes the chosen mechanisms and their parametrization. Section 5 details the experimental setup and software stack. Section 6 presents and analyses the empirical results. Section 7 interprets findings, proposes mitigation, and tests the joint calibration heuristic. Section 8 addresses limitations and future work. Section 9 concludes.

2 Background

This chapter summarizes the theoretical background of FL, the trilemma axes and formalizes the Privacy-Robustness-Performance trilemma.

2.1 Federated Learning

Federated Learning is a machine learning framework comprising a central server and multiple distributed clients. Each client maintains a local dataset to compute updates for the global model. The central server orchestrates the training process by distributing the current global model, collecting client updates, and aggregating them to produce a new global model update. This update is then distributed to clients to initiate the next training round.

The most widely used aggregation method is FedAvg [3], which computes the global update as a weighted average of the client-local model updates.

2.2 Privacy: DP

This chapter introduces the concepts of local and central DP and describes the implementation of local DP in DP-SGD.

2.2.1 Definition and Composition

DP forms a good standard for privacy guarantees for algorithms used in data aggregation. Its degree of protection is defined by the privacy budget ε and the failure probability δ. We use the definition given by Abadi et al. [4].

Definition 1. *A randomized mechanism $M\colon D \to R$ with domain D and range R satisfies (ε, δ)-differential privacy if for any two adjacent inputs $d, d' \in D$ and for any subset of outputs $S \subseteq R$ it holds that*

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta.$$

The privacy budget ε denotes the trade-off between utility and privacy, bounding the tolerated noise addition. Small values ($\varepsilon < 1$) provide the strongest privacy guarantees while larger values ($\varepsilon > 8$) allow for more accurate training.

The failure probability δ, introduced by Dwork et al. [5], denotes the probability that the plain ε-DP is broken and should be chosen smaller than the inverse size of the local dataset.

Central vs. Local DP in FL. In central DP, the server adds noise after aggregating the client updates. This setting assumes a trusted aggregator while protecting the aggregated updates before they are shared with third parties.

In the considered setting, client updates must be protected against inference attacks by the aggregator or adversaries eavesdropping on the client–server communication, requiring local DP. To achieve this, each client adds noise to its update before transmission. Since the noise added

Algorithm 1 Differentially private SGD (Outline)

Input: Examples $\{x_1, \ldots, x_N\}$, loss function $\mathcal{L}(\theta) = \frac{1}{N}\sum_i \mathcal{L}(\theta, x_i)$. Parameters: learning rate η_t, noise scale σ, group size L, gradient norm bound C.

Initialize θ_0 randomly

for $t \in [T]$ **do**

Take a random sample L_t with sampling probability L/N

Compute gradient

For each $i \in L_t$, compute $\mathbf{g}_t(x_i) \leftarrow \nabla_{\theta_t}\mathcal{L}(\theta_t, x_i)$

Clip gradient

$\bar{\mathbf{g}}_t(x_i) \leftarrow \mathbf{g}_t(x_i) / \max\left(1, \frac{\|\mathbf{g}_t(x_i)\|_2}{C}\right)$

Add noise

$\tilde{\mathbf{g}}_t \leftarrow \frac{1}{L}\left(\sum_i \bar{\mathbf{g}}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})\right)$

Descent

$\theta_{t+1} \leftarrow \theta_t - \eta_t \tilde{\mathbf{g}}_t$

Output θ_T and compute the overall privacy cost (ε, δ) using a privacy accounting method.

Figure 1: DP-SGD pseudocode as published by Abadi et al. [4].

by individual clients accumulates during aggregation, local DP requires a higher effective noise magnitude and therefore incurs a greater loss in accuracy.

DP-SGD. DP-SGD is a modification to the standard Stochastic Gradient Descend (SGD) algorithm introduced by Abadi et al. [4] in 2016 to integrate local DP into deep learning frameworks. It enhances SGD by clipping the computed gradient to a fixed norm and adding calibrated (Gaussian) noise before performing an update step. The algorithm pseudocode as proposed by Abadi et al. is shown in figure 1.

2.3 Robustness: Byzantine-Tolerant Aggregation

This section introduces the Byzantine fault model and presents two aggregation schemes designed to mitigate the effects of Byzantine adversaries.

Byzantine Fault Model. Byzantine Fault Tolerance is a property of a distributed system that allows for the system to stay operational while some components fail or act maliciously.

A η-fraction Byzantine adversary means that $\eta\%$ of the clients are acting maliciously. In the conducted experiments, malicious behavior means communicating manipulated gradients to poison the central model, e.g. insert a backdoor, degrade the general performance or enable label flipping attacks.

Krum. Krum [6], proposed by Blanchard et al. in 2017, enhances robustness to Byzantine adversaries by selecting updates closest to their XX neighbors, effectively focusing on the densest cluster of updates of size XX. This ensures the training process is provably Byzantine-tolerant but relies on the assumption that honest updates cluster together, which only holds when data distributions across clients are similar. Under data heterogeneity, this assumption breaks down, leading to performance degradation because honest updates may be discarded if they deviate significantly from the majority.

FLTrust. FLTrust [7] was introduced by Cao et al. in 2020 and mitigates malicious updates through trust scores. These are obtained by calculating the cosine similarity of the local update to an update anchor computed on a small root dataset held by the server. The method relies on the assumption that the server has access to a dataset representing the general data distribution across all clients.

2.4 Communication Efficiency

Another practical limitation especially in mobile applications is the available bandwidth to transmit local updates to the aggregator. We therefore include a method to communicate sparse instead of dense updates.

Top-k Gradient Sparsification. Top-k gradient sparsification [8] reduces communication during gradient transmission by retaining only the k largest gradient components, where k is a fixed fraction of the model dimension d.

Discarding the remaining gradient components inevitably results in information loss, leading to reduced model performance. The extent of this degradation depends on how the optimization information is distributed across the gradient, with models relying on a small number of dominant components generally being less affected than those requiring information from many dimensions.

2.5 The Privacy–Robustness–Performance Trilemma

This section first presents the theoretical lower bound of the trilemma and then discusses its implications for empirical evaluation, motivating efficiency as a fundamental third axis.

Theoretical Lower Bound. Allouah et al. [1] proved that any algorithm simultaneously guaranteeing ε-local DP and robustness against an η-fraction Byzantine adversary incurs a mean estimation error of at least

$$\Omega! \left(\frac{\eta}{\varepsilon^2} + \frac{d}{\varepsilon^2 n} + \eta G^2 \right). \tag{2}$$

This lower bound establishes that the costs of privacy and Byzantine robustness are fundamentally coupled. In particular, the cross-term η/ε^2 shows that these costs interact multiplicatively rather than additively.

Efficiency Axis Extension. The same authors proposed the aggregation schemes SMEA and CAF, which match this theoretical lower bound. However, both methods incur a runtime complexity that scales cubically with the model dimension, rendering them impractical for large-scale models. Furthermore, the runtime of SMEA grows exponentially with the number of tolerated Byzantine adversaries, making it infeasible in realistic federated learning settings.

The term $\frac{d}{\varepsilon^2 n}$ in Equation 2 further relates the model dimension d to the communication cost and achievable estimation error. Consequently, maintaining accuracy for high-dimensional models requires either relaxing privacy guarantees or integrating a larger number of participating clients.

Empirical Evidence of Compounding. In 2021, Guerraoui et al. empirically investigated the question *"Differential Privacy and Byzantine Resilience in SGD: Do They Add Up?"* [9]. They showed that a naive combination of privacy- and robustness-enhancing methods, namely DP-SGD and Minimum-Diameter Averaging (MDA) [10], leads to unfavorable scaling with the model dimensionality and super-additive performance degradation. These findings support the theoretical results of Allouah et al. and reinforce the need to treat efficiency as a fundamental third axis in future experimental studies.

3 Threat Model and Metrics

This chapter maps out the assumed threat model and introduces the metrics used to evaluate the three axes.

3.1 Threat Model

We assume an honest-but-curious central server that adheres to protocol specifications but may attempt to infer private information from transmitted updates, motivating the use of differential privacy (DP) to safeguard client data. The adversary is characterized by a fraction η of malicious clients, capable of arbitrarily manipulating gradients (e.g., through label-flipping attacks, as formalized in Fang et al. [11]). These clients aim to corrupt the global model by injecting adversarial updates. Crucially, we assume no collusion between Byzantine clients and the server, restricting adversarial behavior to local manipulations of gradients. This assumption simplifies the analysis while maintaining practical relevance for decentralized learning systems.

3.2 Evaluation Metrics

As suggested by Allouah et al. [2], every configuration is evaluated simultaneously across all three trilemma axes:

Privacy. The privacy objective is quantified by the assigned (ε, δ)-differential privacy (DP) budget.

Robustness. Robustness is evaluated by the the accuracy gap Δ_{acc} relative to a clean FedAvg baseline. The accuracy gap captures performance degradation caused by adversarial clients, providing a measure of resilience to Byzantine behavior.

Performance. The performance metric includes communication cost (bytes sent per client per round), and wall-clock training time. These metrics collectively assess the trade-off between resource efficiency, and computational feasibility in decentralized learning systems.

3.3 Trilemma Visualization

The values of the three trilemma axes are computed as follows, each yielding a value in the range $[0, 1]$, where 0 denotes no effect and 1 the maximum attainable effect:

$$\text{Privacy} = \max\left(0, \frac{\varepsilon_{\max} - \varepsilon}{\varepsilon_{\max}}\right)$$

$$\text{Robustness} = 1 - \Delta_{acc}$$

$$\text{Efficiency} = \alpha \times \frac{t_{base}}{t} + (1 - \alpha) \times \frac{b_{base} - b}{b_{base}}$$

Here, ε denotes the privacy budget, Δ_{acc} the accuracy gap induced by the Byzantine attack, t the wall-clock training time per client and communication round, and b the number of bytes transmitted per client and communication round. The reference values t_{base} and b_{base} correspond to the wall-clock training time and transmitted bytes of the baseline configuration, respectively.

The weighting parameter α determines the relative contribution of computation time and communication cost to the efficiency axis and can be adjusted to match the requirements of a target deployment. Unless stated otherwise, $\alpha = 0.5$ is used to assign equal weight to both components.

4 Methods

This section briefly describes the implementation of the selected methods for each axis.

4.1 Baseline: FedAvg

FedAvg, introduced by McMahan et al. in 2017 [3], is a widely used aggregation strategy in FL designed to handle non-IID data efficiently. It computes a weighted average of local model updates, with weights determined by the size of each client's local dataset. The paper establishes baselines on standard datasets such as MNIST and CIFAR-10, making FedAvg a foundational reference for evaluating FL methods. Its simplicity and effectiveness in early FL research have cemented its role as a standard baseline for comparative studies.

4.2 Privacy Mechanism: DP-SGD

DP-SGD is implemented according to the algorithm described in Figure 1. Before each optimization step, the ℓ_2 norm of each per-sample gradient is clipped to the threshold C. The clipped gradients are averaged, and Gaussian noise sampled from $\mathcal{N}(0, \sigma^2 C^2 I)$ is added before updating the model parameters, where σ denotes the noise multiplier. The experiments evaluate privacy budgets of $\varepsilon \in 1, 5, 10$ while fixing $\delta = 10^{-5}$.

4.3 Robustness Mechanism: FLTrust

The root dataset for the central server was obtained by randomly sampling a parametrized number of data points, uniformly distributed over all classes, from the training set.

The trust score for weighting the local contributions is calculated as the cosine similarity to the update vector that was computed from the root data set as depicted in figure 2.

4.4 Performance Mechanism: Top-k Sparsification

Top-k sparsification is implemented as a custom client-side wrapper that retains only the $k\%$ gradient components with the largest magnitude, where $k \in 1, 10, 50$, and transmits their corresponding indices alongside the retained values. Consequently, the number of transmitted bytes is not reduced exactly to $k\%$ of the original update size because the component indices must also be communicated. Nevertheless, a substantial reduction in communication volume is expected, particularly for small values of k.

4.5 Mechanism Interaction Effects

In this section we outline the expected compounding effects.

Performance. We hypothesize that the clipping and calibrated noise introduced by DP-SGD impair the assignment of meaningful trust scores during FLTrust aggregation, thereby slowing convergence and reducing the final model accuracy beyond the individual cost of DP-SGD. Furthermore, DP-perturbed updates are expected to be more sensitive to Top-k sparsification, as the added noise may increase the magnitude of gradient components that would otherwise be discarded. Consequently, combining multiple mechanisms is expected to incur a super-additive loss in accuracy.

Efficiency. We expect the noise addition required for DP-SGD to introduce a noticeable computational overhead on the client side, while FLTrust incurs a slight overhead on the server side. In contrast, Top-k sparsification is not expected to reduce the overall training time in our experimental setup, as all clients are simulated on a single machine and communication latency is therefore negligible. However, the number of bytes transmitted per training round is expected to decrease proportionally to the sparsification ratio.

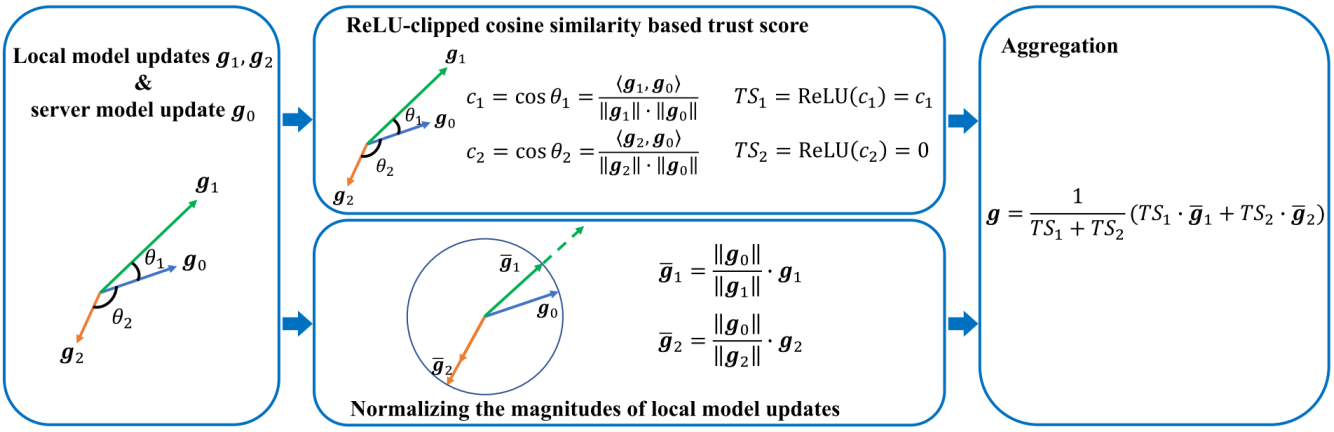

Figure 2: Global aggregation rule for FLTrust as introduced by Cao et al. [7].

5 Experimental Setup

5.1 Experimental Grid

The experimental grid is described in table 1.

Identifier	DP-SGD	FLTrust	Top-k
Base			
DP	x		
FLTrust		x	
TOPK			x
DP+FLTrust	x	x	
DP+TOPK	x		x
FLTrust+TOPK		x	x
ALL	x	x	x

Table 1: Identifiers and applied methods for the different experimental setups.

5.2 Datasets

MNIST. The MNIST dataset consists of 70,000 grayscale images of handwritten digits with a resolution of 28 × 28 pixels, each labelled with the corresponding digit in the range [0,9]. It is divided into a training set of 60,000 samples and a test set of 10,000 samples. Owing to its simplicity, the dataset is widely used to benchmark image classification models and verify their functionality before extending them to more complex datasets.

CIFAR-10. CIFAR-10 is a benchmark dataset for object classification comprising 60,000 RGB images with a resolution of 32 × 32 pixels, categorized into 10 classes. It is divided into 50,000 training samples and 10,000 test samples and is widely used for training and benchmarking object classification networks.

5.3 Learning and attack settings

For a training setup with n clients, the dataset is partitioned into $n+1$ disjoint subsets: one uniformly sampled root dataset for FLTrust and n randomly sampled client datasets. We assume an IID data distribution across all clients and therefore do not introduce artificial label imbalances. The resulting label distribution for the MNIST experiments is shown in Figure 7.

The label-flipping attack is implemented by swapping the labels of the digits "3" and "7" in the MNIST dataset and the classes "cats" and "dogs" in the CIFAR-10 dataset for the malicious clients. The resulting gradients are subsequently scaled by the attack scale parameter before being transmitted to the central server to amplify the attack.

5.4 Parameters

Static. The following parameters are constant across all experiment runs.

- The client learning rate is fixed at 0.01.
- The number of gradient descend steps taken by each client per round is set to 10, due to the results of analyzing the effect on the model accuracy and evolution of the trust scores described in section 6.4.
- The initial seed for random processes is fixed to 42 to ensure comparability across runs. Randomizers required to vary between rounds, like training set shuffle or DP noise generation, additionally take the round number as input.
- The attack type is fixed to flip the labels of digits 3 and 7 on the MNIST dataset.

- DP failure probability δ is fixed to 1e-5.
- The root dataset size for FLTrust is fixed to 2000 samples. This is very large compared to the size of 100 samples proposed by the authors.

Dynamic. The following parameters are varied across experiment runs to investigate their influence on the results.

- Number of clients
- Number of training rounds
- Byzantine fraction for label flipping attack
- Attack scale for label flipping attack
- DP budget $\varepsilon \in [1; 5; 10]$
- Top-K sparsification threshold $k \in [1\%; 10\%; 50\%]$

5.5 Software Stack

- **Flower** (flwr) [12] — FL orchestration, client/server simulation, strategy interface.
- **Opacus** [13] — DP-SGD
- **FLTrust** [7] — FLTrust defense implementation.
- **Custom** — Top-k gradient sparsification wrapper.

[Software versions; hardware (GPU/CPU); random seed management; number of independent runs per configuration.]

5.6 Reproducibility

[Pointer to open-source repository; instructions to replicate all 8 configurations; fixed random seeds; containerization (Docker/Conda environment file).]

6 Results

This section summarizes the experimental results. The final round accuracy over all configurations is reported in Figure 8.

6.1 Baseline Reproduction (RQ1 – Sanity Check)

Training a clean FedAvg model using only honest clients on the MNIST dataset yielded a final accuracy of 98%, slightly below the 99% reported in the original FedAvg paper [3].

The original FLTrust paper reported an accuracy of 99.6% on the MNIST dataset for both clean FedAvg training and training under attack when FLTrust was enabled. In our experiments, enabling FLTrust achieved a final accuracy of 98%, matching the performance of the clean FedAvg baseline. Although the absolute accuracy is lower than that reported by the original authors, our results likewise indicate that FLTrust effectively mitigates the label-flipping attack under the considered experimental setting.

6.2 Isolation: Individual Mechanism Costs (RQ1)

This chapter analyzes the cost and effects of the isolated mechanisms compared to the baseline.

6.2.1 DP-SGD Cost

Training time. The replacement of pure SGD with DP-SGD on the clients results in an increase in training time. This effect is stronger for stricter privacy budgets as depicted in Figure 13.

Final accuracy. Regarding the model performance, all conducted experiments report an accuracy loss of ≈ 0.2 compared to the baseline (see Figures 8 and 12). Contrary to our expectations, the assigned privacy budget does not have a major influence on the accuracy besides the general degradation observed when applying DP-SGD: We achieve similar performance for all $\varepsilon \in [1; 5; 10; 25]$. Figure 9 shows a slight variation in the early rounds but overall a similar evolution for all values of ε.

6.2.2 FLTrust

Effect. Enabling FLTrust improves model accuracy and mitigates the effects of the label-flipping attack, as shown in Figures 4 and 5. In the baseline model, the targeted digits 3 and 7 are consistently misclassified as each other, whereas enabling FLTrust largely restores correct predictions. The improvement becomes more pronounced with

increasing Byzantine fractions and attack scales. The results shown were obtained with a Byzantine fraction of $\eta = 40\%$ and an attack scale of 2.0.

Cost. The replacement of FedAvg aggregation with FLTrust did not yield a noticeable impact on the wall clock training time.

6.2.3 Top-k Sparsification

Effect. Applying Top-k sparsification yields the expected savings in bandwidth, modeled here by measuring the bytes transferred per client per round. The results are visualized in Figure 3, confirming a reduction of $100 - 2k\%$. The factor of two comes from the necessity to transfer also the indices of the transferred bytes, rendering all $k \geq 50$ impractical for saving bandwidth.

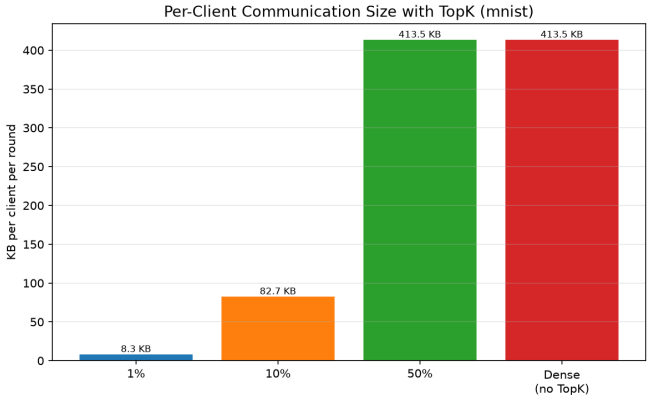

Figure 3: Bytes transferred per client per round for settings with different Top-k sparsification thresholds.

Cost. We do not observe a significant decline in performance when applying Top-k sparsification to the MNIST training process. Cutting down to transferring only the top 1% of the gradient yields a slight loss in accuracy, but not a noticeable effect.

One possible explanation for the minimal effect of the sparsification is that MNIST is a relatively simple dataset for which the classification depends on a very limited number of features.

6.3 Combination: Pairwise and Full Interactions (RQ2)

This section reports the combinatory effects observed when pairing multiple mechanisms. The full results are available in table 3.

6.3.1 DP + Robustness

Cost. Combining DP-SGD with FLTrust results in a substantial decline in model accuracy when training full epochs per round, reducing performance to only slightly above random guessing. Figure 6 shows the confusion matrix of the final model, revealing that it predicts only two classes.

These results are consistent with the super-additive interaction between privacy and Byzantine robustness derived by Allouah et al. Intuitively, the noise introduced for differential privacy causes client updates to deviate from the anchor update computed on the root dataset, resulting in uniformly low trust scores. This behavior is reflected in the second chart of Figure 11.

When reducing the local training steps to 10 batches per round (final accuracies available in Figure 12), the decline is surprisingly soft, surpassing the results of DP alone and showcasing the same accuracy gap against the FLTrust-only results as the DP-only config did against the base configuration Contrary to our expectations, for the combination of FLTrust and DP, stricter privacy guarantees ($\varepsilon = 1$) led to even better accuracy than more relaxed ones ($\varepsilon = 10$), resulting in accuracy gaps down to $\Delta_{acc} = 0.05$ compared to the honest FedAvg setting.

Influence of the ε value. Although model performance varies slightly with ε, even the relaxed setting of $\varepsilon = 25$ does not achieve an accuracy above 0.2. The most pronounced effect is shown in Figure 10, which illustrates the accuracy throughout training. While configurations with $\varepsilon \in 5, 10, 25$ reach a peak accuracy of approximately 0.2 after 35, 25, and 15 training rounds, respectively, the configuration with $\varepsilon = 1$ never exceeds an accuracy of 0.12, indicating substantially slower convergence.

6.3.2 DP + Compression

Applying Top-k sparsification to updates perturbed by DP noise significantly affects model performance, an effect that is not observed in the Top-k-only configuration. While $k = 50\%$ does not introduce additional degradation beyond that caused by DP-SGD alone, smaller values of k substantially reduce the final model accuracy, with $k = 1\%$ reaching a peak accuracy of only 0.2 when training full epochs and 0.27 when reducing that to 10 SGD steps per round.

As in the DP-only configuration, varying the privacy budget ε has no noticeable impact on model accuracy.

6.3.3 Robustness + Compression

Combining FLTrust with Top-k sparsification largely preserves the individual effects of both methods. Transmitting only 1% of the gradient results in the same slight reduction in accuracy observed in the Top-k-only configuration, while FLTrust provides a comparable improvement in robustness to that achieved with dense updates. Figure 11 shows that the trust scores exhibit a similar evolution to the dense-update setting.

6.3.4 Full Combination (DP + Robustness + Compression)

For full local-epoch training, combining all three methods reduces model performance to a level comparable to that of the DP+FLTrust configuration. The final accuracy ranges between 0.12 and 0.18, only marginally exceeding random guessing, and the resulting confusion matrix closely resembles that of the DP+FLTrust setting.

Reducing the local training to 10 steps per communication round, however, produces markedly different results. The performance degradation previously observed for Top-1% sparsification in the DP+Top-k configuration no longer occurs when FLTrust is added. Furthermore, the trend of stronger privacy guarantees yielding higher accuracy persists across all evaluated values of k. For a configuration with 60 clients, an accuracy of 88% is achieved with a privacy budget of $\varepsilon = 1$ while transmitting only 1% of the locally computed gradient components to the server.

6.4 Trust and Privacy: FlTrust scores (RQ3)

This section reports the findings related to the trust scores computed during the FLTrust aggregation.

Development of trust scores. Figure 11 illustrates the evolution of the trust scores throughout training. The first chart, corresponding to the isolated FLTrust configuration, shows a clear separation between honest and malicious clients. Honest clients start with trust scores of approximately 0.9, which gradually decrease after roughly 15 rounds before stabilizing around 0.3. Malicious clients start near 0.7, exhibit a brief increase, and then decline sharply to 0 after approximately 25 training rounds.

One possible explanation for the gradual decline in the trust scores of honest clients is that differences between the local training datasets become increasingly influential once the model has learned the general structure of the data. Consequently, honest updates remain broadly aligned with the root dataset but become progressively more diverse, whereas the poisoned and scaled updates of malicious clients deviate from the server update anchor and are therefore assigned trust scores close to zero.

Interaction of trust scores and batches processed per round. Table 2 summarizes the effect of the number of locally processed batches per communication round on the final model accuracy and the evolution of the trust scores assigned to honest and malicious clients. Processing an entire local epoch per round yields the highest model accuracy but causes the trust scores of honest clients to converge toward zero during later training stages. In contrast, processing only a single batch per round, as proposed by the original authors, results in substantial overlap between the trust scores of honest and malicious clients, reducing the effectiveness of FLTrust and leading to significantly lower accuracy.

Processing 5–20 batches per round preserves a clear separation between honest and malicious trust scores while maintaining high model accuracy. Among these configurations, 10 batches per round provide a favorable trade-off and are therefore used in the second experimental setup. For the full-epoch variant, the training rounds are limited to 50 to avoid the collaps of the trust scores for honest clients.

7 Discussion

In this section, we compare the experimental results with the hypotheses formulated in Section 4.5.

Influence of the Optimization Schedule on the Interaction of Differential Privacy and FLTrust. Contrary to our hypothesis, the combination of DP-SGD and FLTrust did not generally result in a super-additive decline in performance, but only showed the expected degradation when training in full epochs. For this initial setting, we observed the expected behavior: The trust scores of both honest and malicious clients collapse during the early stages of training, preventing further convergence and limiting the model accuracy to little more than random guessing.

Local steps / round	Rounds	Accuracy	Honest trust score (late training)	Malicious trust score
Full epoch	300	0.986	0.016 (collapsed to 0 after $\approx$ 100 rounds)	0.000
1	300	0.782	0.450 (high, strongly fluctuating)	0.325 (overlaps with honest)
5	600	0.946	0.02–0.22 (stable, non-collapsing)	0.000
10	600	0.969	0.05–0.19 (stable, non-collapsing)	0.000
20	600	0.968	0.02–0.21 (stable, non-collapsing)	0.000

Table 2: Summary of the evolution of trust scores for honest and malicious clients during late training (after 250+ rounds).

One possible explanation is that, during full local epochs, the client models drift apart, causing the updates to naturally become more dissimilar. The addition of privacy-preserving noise further amplifies this drift, resulting in the collapse of the trust scores. With fewer local optimization steps, client drift remains limited, preserving a higher degree of similarity between honest client updates and the FLTrust anchor update. Consequently, the additional perturbation introduced by differential privacy is no longer sufficient to destroy the trust-score assignment.

These findings suggest that the optimization schedule constitutes an important moderating factor in the interaction between differential privacy and Byzantine robustness. While the theoretical lower bound remains valid, the practical severity of the interaction appears to depend strongly on the amount of local optimization performed between communication rounds.

Interaction of DP and FLTrust. Enabling FLTrust led to an inversion of the expected decline in performance for stronger privacy guarantees. We did not find a solid explanation for this phenomenon during our experiments and suggest that future work investigate this behavior further. This observation indicates that the interaction between differential privacy and Byzantine robustness may be more complex than suggested by the theoretical lower bound alone.

Interaction of Sparsification and DP. As expected, applying aggressive sparsification to gradients perturbed by privacy-preserving noise degrades performance super-additively because the transmitted dimensions are no longer necessarily those carrying the most useful information, but may instead include components whose magnitudes were increased by the added noise.

Interaction of FLTrust and Top-k Sparsification. Sparsifying the gradients prior to transmission did not have a noticeable impact on either the final model accuracy or the trust scores computed during aggregation. A possible explanation is that cosine similarity is largely insensitive to changes in vector magnitude. Since Top-k sparsification preserves the dominant gradient components, the direction of the client updates remains sufficiently similar for FLTrust to assign meaningful trust scores.

Evolution of Trust Scores. The temporal behavior of the trust scores of honest and malicious clients has received little attention in the literature. One possible explanation for the observed dynamics is that, during the early stages of training, the model learns the general structure of the data, resulting in relatively similar client updates. Once the model has captured these global patterns, the honest updates increasingly reflect the characteristics of the individual local datasets and gradually drift apart, causing their trust scores to decrease. At the same time, the influence of the label-flipping attack becomes increasingly dominant in the malicious updates, leading to a collapse of their trust scores.

8 Limitations and Future Work

8.1 Limitations

The experiments were conducted under several simplifying assumptions to ensure a manageable experimental setup while establishing a baseline for future research.

Simple models and datasets. The selected datasets and models facilitate comparison with existing work but may not adequately represent the data characteristics and model architectures encountered in practical FL applications, such as medical, financial, or transformer-based systems.

Limited method coverage. Only a single mechanism and, where applicable, a single attack type were evaluated for each axis of the trilemma. Furthermore, the baseline for the membership inference attack success rate was approximated using a shadow model, and the evaluated privacy budgets were limited to a small ε-grid to balance computational feasibility with analytical rigor.

FLTrust root dataset size. The comparatively large root dataset may overestimate the effectiveness of the FLTrust defense mechanism. The experiments should therefore be extended to investigate the influence of smaller root datasets, as proposed in the original work.

Stateless clients. The clients do not preserve state across communication rounds. In combination with Top-k sparsification, this prevents small but consistent gradient updates from accumulating over time.

Introducing client-side state would require honest clients to remain synchronized with the global model. While this assumption is realistic for distributed training, it is difficult to justify in federated learning. Alternatively, maintaining the state on the server would reduce the benefits of sparsification and introduce additional privacy concerns.

Privacy budget schedule. The privacy budget was allocated uniformly across training rounds. However, recent work by Kiani et al. [14] suggests that a time-adaptive allocation yields improved privacy–utility trade-offs.

8.2 Future Work

In this section, we derive directions for future research from the open findings and limitations of this work.

Investigate the Interaction of DP and FLTrust. Future work should investigate the interaction between DP and FLTrust in greater detail, particularly the observed inversion of the influence of the privacy budget on model performance, to determine whether this behavior can be reproduced and quantified in more complex experimental settings. Additionally, the evolution of the trust scores should be analyzed more closely to better understand the mechanisms underlying the observed behavior.

Trust Scores as a Regularization Mechanism. The gradual decay of the trust scores once the global model has been established suggests that they may capture increasing specialization of the local client models. Future work could therefore investigate whether these trust scores can be exploited as a regularization mechanism to reduce overfitting to individual client datasets.

Broader Experimental Setup. Future work should extend the detailed analysis to CIFAR-10 and more challenging datasets that better reflect practical federated learning applications.

Furthermore, the experimental setup should be expanded to encompass a broader range of aggregation methods, such as Krum, SMEA, and CAF, as well as stronger adversarial scenarios, including the model poisoning attacks proposed by Fang et al. [11]. Exploring reparameterization techniques such as the Low-Rank Adaptation method proposed by Liu et al. [15] could further improve the understanding of the trade-offs between efficiency and privacy.

Finally, future work should investigate alternative threat models, including malicious servers and collusion between malicious clients and the server, as well as non-IID data distributions through artificially skewed client datasets. Together, these extensions would provide a more comprehensive understanding of the privacy–robustness–performance trilemma and help determine whether the interactions observed in this work generalize to more realistic federated learning settings.

9 Conclusion

[Restate research questions and answers; key quantitative finding (e.g. full combination degrades accuracy by X% beyond sum of individual costs); mitigation heuristic reduces compounding by Y% without formal guarantee loss; open-source pipeline as contribution to the community.]

References

[1] Y. Allouah, R. Guerraoui, N. Gupta, R. Pinot, and J. Stephan, "On the privacy-robustness-utility trilemma in distributed learning," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, 2023. [Online]. Available: https://arxiv.org/abs/2302.04787

[2] Y. Allouah, R. Guerraoui, and J. Stephan, "Balancing privacy, robustness, and efficiency in machine learning," *arXiv preprint*, vol. arXiv:2312.14712, 2023. [Online]. Available: https://arxiv.org/abs/2312.14712

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282.

[4] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 308–318.

[5] C. Dwork, K. Kenthapadi, F. McSherry, I. Mironov, and M. Naor, "Our data, ourselves: privacy via distributed noise generation," ser. EUROCRYPT'06. Berlin, Heidelberg: Springer-Verlag, 2006, p. 486–503. [Online]. Available: https://doi.org/10.1007/11761679_29

[6] P. Blanchard, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 119–129.

[7] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Network and Distributed System Security Symposium (NDSS)*, 2021. [Online]. Available: https://arxiv.org/abs/2012.13995

[8] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017, pp. 440–450.

[9] R. Guerraoui, N. Gupta, R. Pinot, S. Rouault, and J. Stephan, "Differential privacy and byzantine resilience in sgd: Do they add up?" 2021. [Online]. Available: https://arxiv.org/abs/2102.08166

[10] E.-M. El-Mhamdi, R. Guerraoui, A. Guirguis, L. N. Hoang, and S. Rouault, "Genuinely distributed byzantine machine learning," 2020. [Online]. Available: https://arxiv.org/abs/1905.03853

[11] M. Fang, X. Cao, J. Jia, and N. Z. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proceedings of the 29th USENIX Security Symposium*, 2020, pp. 1605–1622.

[12] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint*, vol. arXiv:2007.14390, 2020. [Online]. Available: https://arxiv.org/abs/2007.14390

[13] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, "Opacus: User-friendly differential privacy library in PyTorch," *arXiv preprint*, vol. arXiv:2109.12298, 2021. [Online]. Available: https://arxiv.org/abs/2109.12298

[14] S. Kiani, N. Kulkarni, A. Dziedzic, S. Draper, and F. Boenisch, "Differentially private federated learning with time-adaptive privacy spending," 2025. [Online]. Available: https://arxiv.org/abs/2502.18706

[15] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, "Differentially private low-rank adaptation of large language model using federated learning," *ACM Transactions on Management Information Systems*, vol. 16, 2025.

10 APPENDIX

10.1 Experimental run on MNIST dataset with 50 rounds of full epochs trained, $n = 50, \eta = 40\%$, attack scale = 2.0

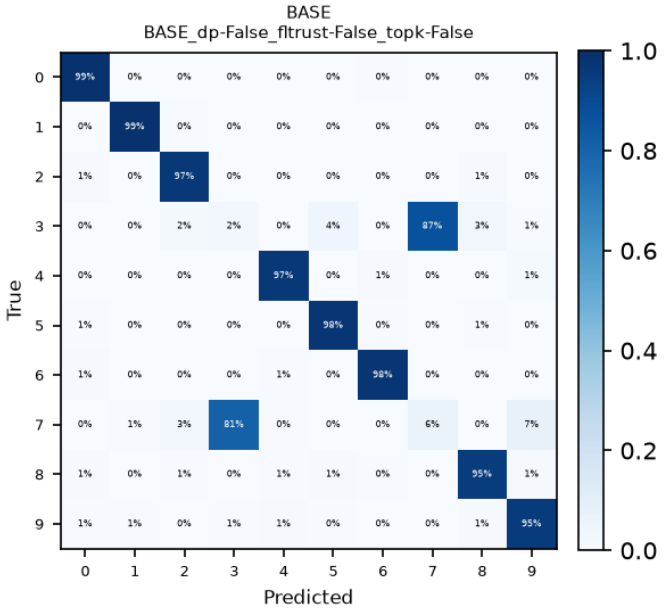

Figure 4: Final confusion matrices for standard FedAvg .

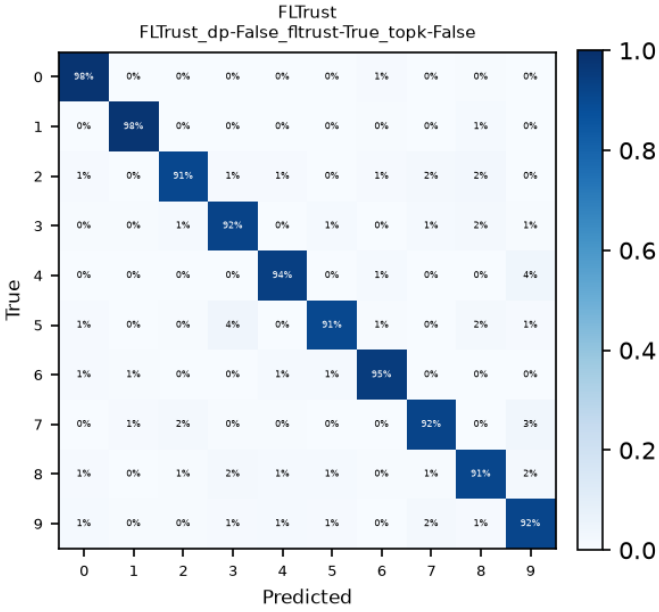

Figure 5: Final confusion matrices for FLTrust config.

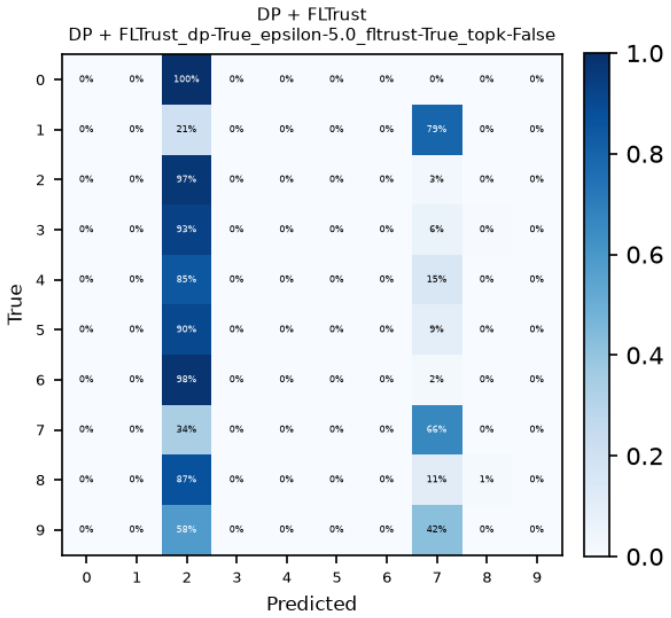

Figure 6: Final confusion matrix for the DP+FLTrust configuration.

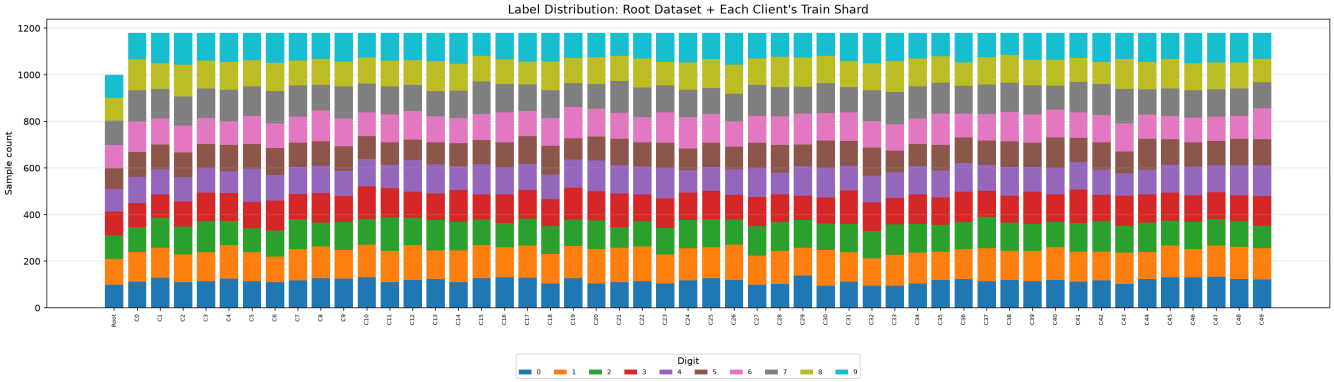

Figure 7: Label distribution of the root dataset used for FLTrust and the client training datasets. The root dataset was sampled uniformly across all classes, while the client datasets were sampled randomly.

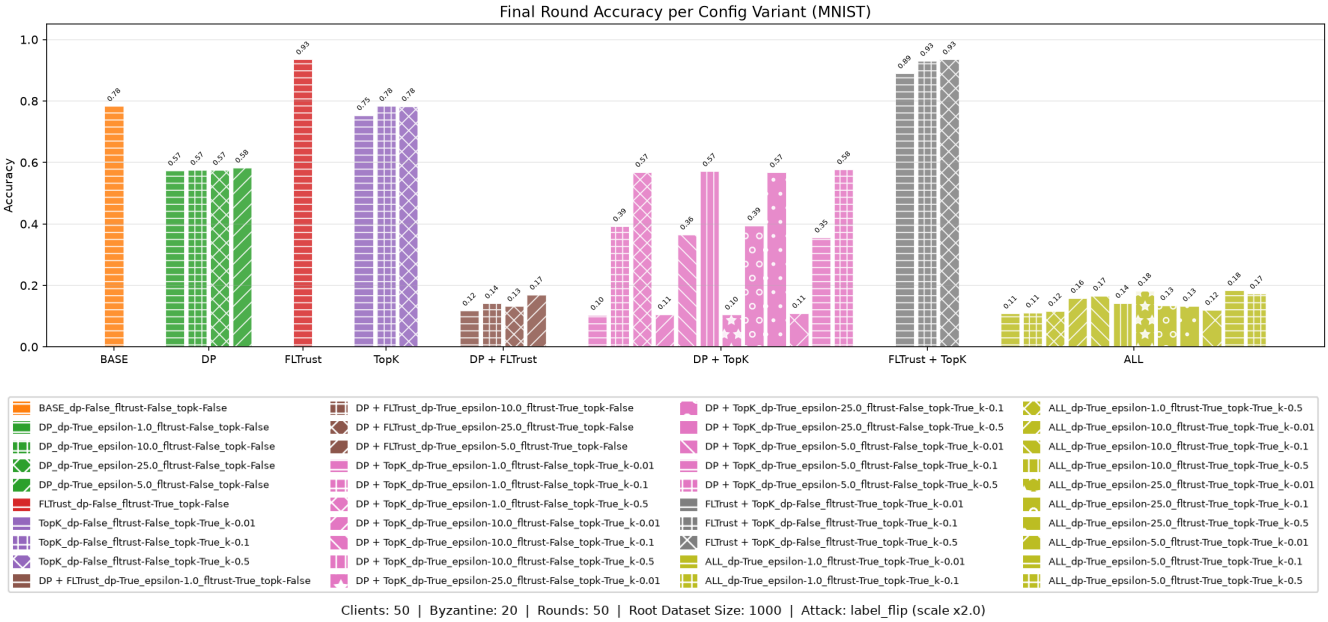

Figure 8: Final accuracy achieved on all different configurations.

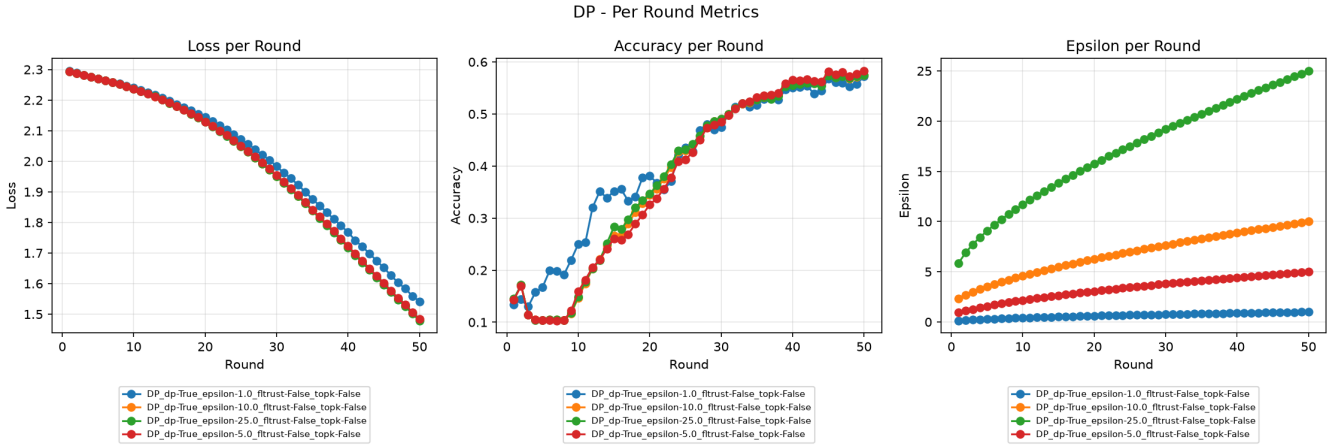

Figure 9: Accuracy over training rounds for DP-only configuration.

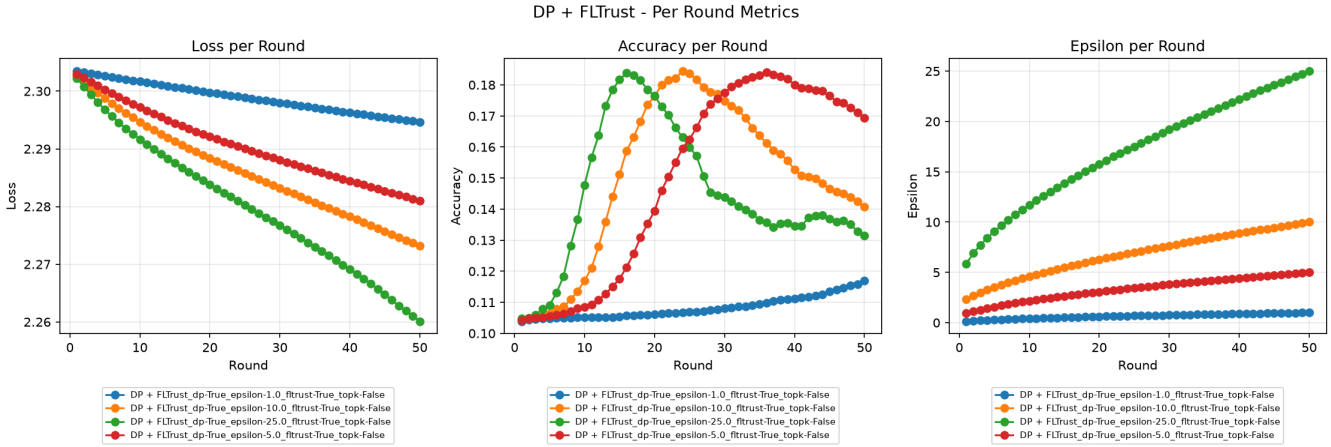

Figure 10: Accuracy per training round for DP+FLTrust configuration.

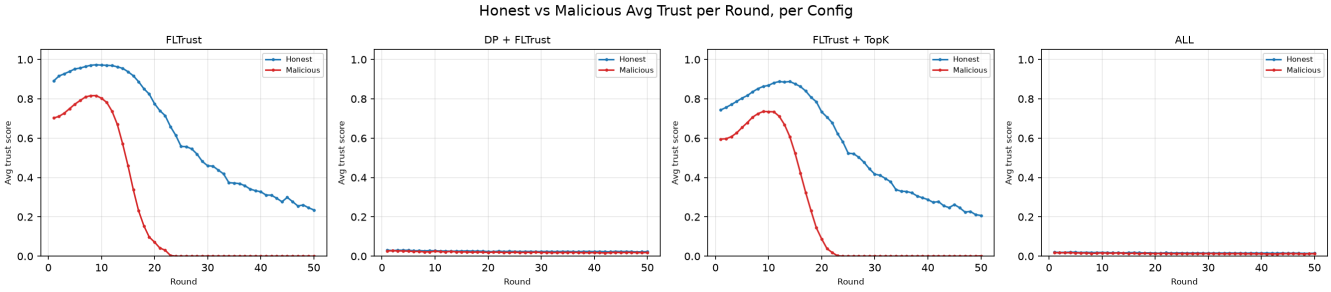

Figure 11: The line charts depict the average trust scores of honest and malicious clients during the training rounds. While for settings where DP is enabled the scores are consistently equally low, the settings without DP-related noise addition show a clear distinction.

10.2 Experimental run on MNIST dataset with 500 rounds of 10 SGD steps each, $n \in \{10, 30, 60\}, \eta = 20\%$, attack scale = 2.0

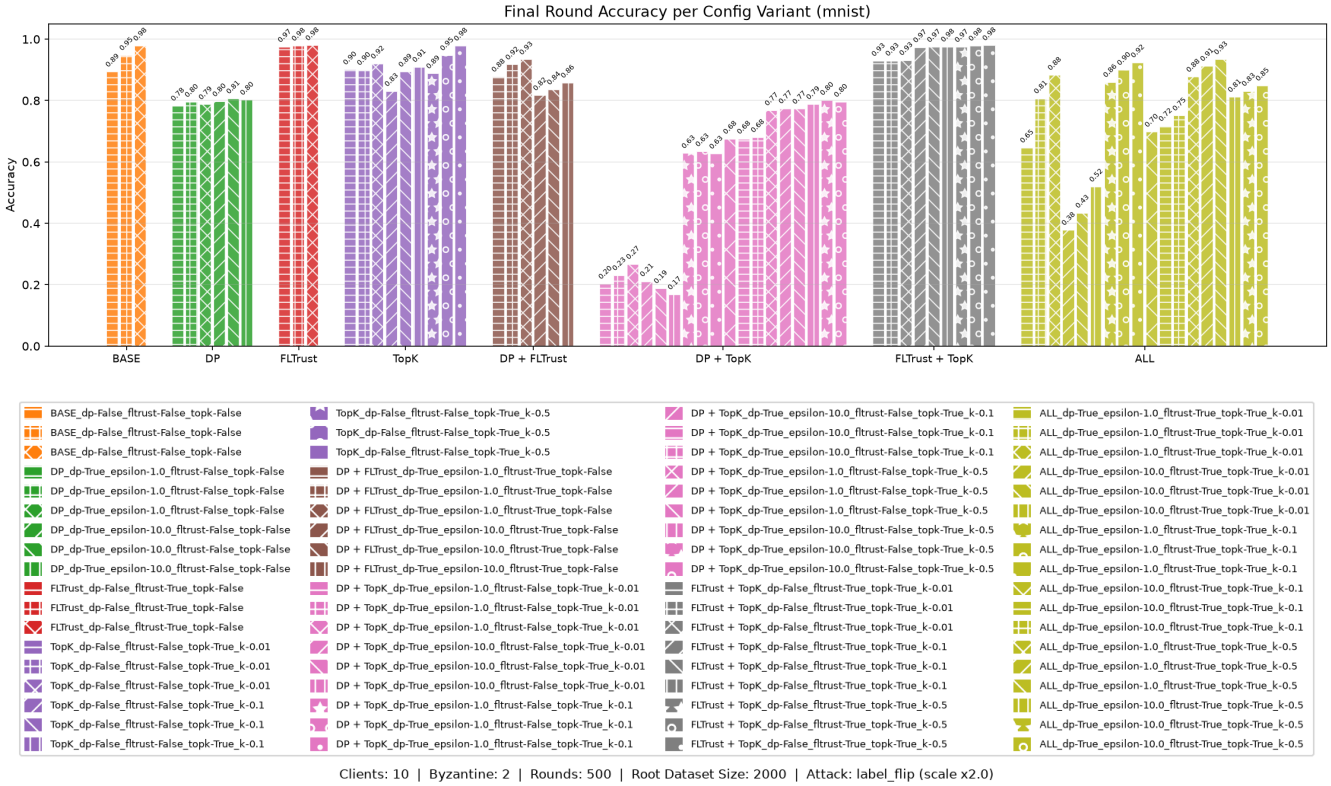

Figure 12: Final accuracy achieved on all different configurations.

Configuration	Clients	ε	Top-k	Δ Accuracy	Final Accuracy
BASE	10	–	–	0.0858	0.8936
	30	–	–	0.0319	0.9475
	60	–	–	0.0008	0.9785
DP	10	1	–	0.1964	0.7830
	30	1	–	0.1844	0.7950
	60	1	–	0.1919	0.7874
	10	10	–	0.1824	0.7970
	30	10	–	0.1731	0.8063
	60	10	–	0.1773	0.8020
FLTrust	10	–	–	0.0055	0.9739
	30	–	–	0.0012	0.9782
	60	–	–	-0.0005	0.9798
TopK	10	–	0.01	0.0785	0.9009
	30	–	0.01	0.0802	0.8992
	60	–	0.01	0.0594	0.9199
	10	–	0.1	0.1490	0.8304
	30	–	0.1	0.0859	0.8935
	60	–	0.1	0.0701	0.9092
	10	–	0.5	0.0913	0.8881
	30	–	0.5	0.0324	0.9470

Continued on next page

Table 3 – continued from previous page

Configuration	Clients	ε	Top-k	$\Delta_{Accuracy}$	Final Accuracy
	60	–	0.5	0.0009	0.9784
DP + FLTrust	10	1	–	0.1022	0.8772
	30	1	–	0.0624	0.9170
	60	1	–	0.0458	0.9335
	10	10	–	0.1627	0.8167
	30	10	–	0.1439	0.8355
	60	10	–	0.1216	0.8577
DP + TopK	10	1	0.01	0.7768	0.2026
	30	1	0.01	0.7490	0.2304
	60	1	0.01	0.7121	0.2672
	10	10	0.01	0.7689	0.2105
	30	10	0.01	0.7912	0.1882
	60	10	0.01	0.8113	0.1680
	10	1	0.1	0.3513	0.6281
	30	1	0.1	0.3451	0.6343
	60	1	0.1	0.3529	0.6264
	10	10	0.1	0.3043	0.6751
	30	10	0.1	0.3041	0.6753
	60	10	0.1	0.2991	0.6802
	10	1	0.5	0.2118	0.7676
	30	1	0.5	0.2055	0.7739
	60	1	0.5	0.2067	0.7726
	10	10	0.5	0.1914	0.7880
	30	10	0.5	0.1779	0.8015
	60	10	0.5	0.1838	0.7955
FLTrust + TopK	10	–	0.01	0.0510	0.9284
	30	–	0.01	0.0500	0.9294
	60	–	0.01	0.0480	0.9313
	10	–	0.1	0.0065	0.9729
	30	–	0.1	0.0053	0.9741
	60	–	0.1	0.0040	0.9753
	10	–	0.5	0.0054	0.9740
	30	–	0.5	0.0011	0.9783
	60	–	0.5	-0.0004	0.9797
ALL	10	1	0.01	0.3330	0.6464
	30	1	0.01	0.1729	0.8065
	60	1	0.01	0.0962	0.8831
	10	10	0.01	0.6017	0.3777
	30	10	0.01	0.5454	0.4340
	60	10	0.01	0.4603	0.5190
	10	1	0.1	0.1195	0.8599
	30	1	0.1	0.0802	0.8992
	60	1	0.1	0.0556	0.9237
	10	10	0.1	0.2817	0.6977
	30	10	0.1	0.2627	0.7167
	60	10	0.1	0.2285	0.7508
	10	1	0.5	0.1026	0.8768

Continued on next page

Table 3 – continued from previous page

Configuration	**Clients**	ε	**Top-k**	$\Delta_{Accuracy}$	**Final Accuracy**
	30	1	0.5	0.0670	0.9124
	60	1	0.5	0.0453	0.9340
	10	10	0.5	0.1681	0.8113
	30	10	0.5	0.1485	0.8309
	60	10	0.5	0.1312	0.8481

Table 3: Final and delta accuracy (relative to the no-attack BASE run) across all evaluated configurations. Unless noted otherwise, all runs use $R_\ell = 10$ local epochs, a Byzantine client fraction of 0.2, and 500 communication rounds.

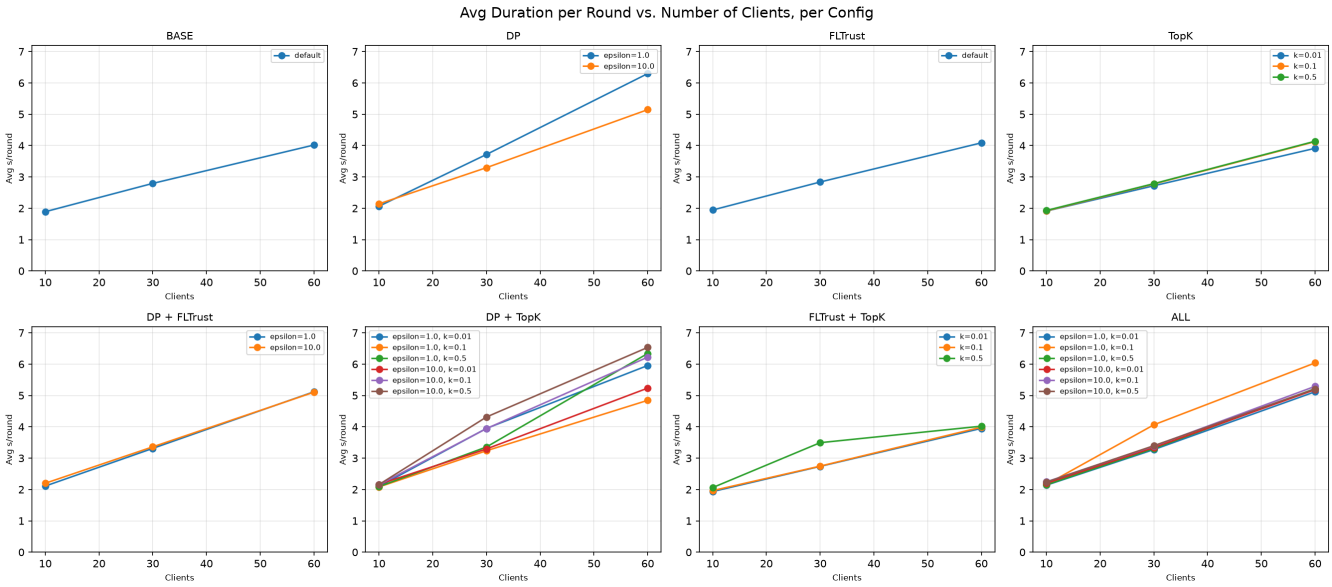

Figure 13: The average training duration per round for varying numbers of clients.